Implement Auto-retry Promises

Company: Amazon · **Tech stack:** JavaScript (ES2022, runs in Node and the browser) · **Time limit:** 45 minutes

> Write `retry(fn, options)` that calls an async function and, if it fails, calls it again — up to N times, with a wait between attempts — and resolves with the first success or rejects with the final error.

PART A: Thought process

General steps for any machine coding task

1. **Clarify** — restate the problem, ask questions, write down assumptions.
2. **List use cases** — every input and the expected output. This becomes the test list.
3. **Design the shape** — modules / functions and one-line responsibility each.
4. **Design the state** — what the code must remember while running.
5. **Define contracts** — function signature, options, return value, errors.
6. **Trace one flow end to end** — before writing code.
7. **Find the hard parts** — solve them on paper first.
8. **Build in layers** — simplest working version first, then one feature at a time, testing each layer.
9. **Cover states and edge cases.**
10. **Wrap up** — trade-offs and what you would do with more time.

Time budget (45 minutes)

Minutes	Activity
0–5	Clarify requirements, agree on the signature
5–10	Use cases, contract, hard parts on the whiteboard
10–20	Basic version: N attempts with fixed delay, tested
20–32	Exponential backoff, `shouldRetry`, `onRetry`, validation
32–40	Cancellation (`AbortSignal`) and per-attempt timeout
40–45	Run the test cases out loud, discuss trade-offs

Think out loud

Say every decision before you type it: "*I take a function, not a promise, because a promise has already started and cannot be re-run.*" The interviewer grades your reasoning as much as your code. If you get stuck, say what you are trying and what you expect to see.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Accept a function that returns a promise (or a value, or throws).
- Call it; if it resolves, resolve with that value.
- If it rejects, wait, then call it again — up to `retries` extra times.
- If every attempt fails, reject with the error of the **last** attempt.
- Configurable wait: fixed delay, or exponential backoff with an upper cap.
- Let the caller decide which errors are worth retrying (`shouldRetry`).
- Notify the caller before each retry (`onRetry`) — for logging / metrics.

Non-functional requirements

Concern	What it means here
Performance	No busy waiting; one timer at a time; no stack growth across attempts.
Scalability	Jitter so 10,000 clients do not retry at the same millisecond (thundering herd). Cap on delay.
Error handling	Synchronous throws treated like rejections; no unhandled rejections; original error preserved.
Usability	Sensible defaults: `retry(fn)` works with no options. Clear `TypeError` for wrong input.
Reusability	Not tied to `fetch` — works for any async operation (DB call, file read, payment API).
Extensibility	Options object, so new features (timeout, abort) don't change the signature.

Clarifying questions to ask

1. Is the input a **function returning a promise**, or a promise itself? *(It must be a function — a promise can't be re-run.)*
2. Does `retries: 3` mean 3 attempts total or 3 retries after the first try (4 calls)?
3. Fixed delay or exponential backoff? Should there be a maximum delay?
4. Should every error be retried, or only some (e.g. network / 5xx, but not 400 / 401)?
5. On final failure: reject with the last error, the first error, or all errors?
6. Do we need cancellation and a timeout per attempt?

Assumptions we will build with

- Input is a function `fn(attempt, signal)` . Its return value may be a promise or a plain value.
- `retries` = number of **extra** attempts → total calls = `retries + 1` . Default `3` .
- Default delay `1000 ms` , `factor: 2` (exponential), `maxDelay: 30 s` . `factor: 1` gives a fixed delay.
- Reject with the **last** error (it is the most recent state of the world).

- Optional: `jitter`, `timeout` per attempt, `signal` to cancel, `shouldRetry`, `onRetry`.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
`fn` resolves on first call	Resolves with its value; `fn` called once; no delay
`fn` fails 2 times, then resolves, `retries: 3`	Resolves with value; `fn` called 3 times
`fn` always fails, `retries: 2`	Rejects with the error of the 3rd call; `fn` called 3 times
`retries: 0`	Exactly one attempt, no retry
`fn` throws synchronously	Treated like a rejection — retried
`fn` returns a non-promise value	Resolves with that value
`shouldRetry` returns `false` (e.g. HTTP 400)	Rejects immediately with that error
`delay: 10, factor: 2, maxDelay: 30`	Waits 10, 20, 30, 30 ms between attempts
`jitter: true`	Each wait is random in `[0, computed delay]`
`onRetry` given	Called with `(error, attempt, wait)` before each wait
`signal` aborted during a wait	Rejects at once with `signal.reason`; timer cleared; no more calls
`signal` already aborted	Rejects; `fn` never called
`timeout: 5000`, attempt hangs	That attempt fails with `TimeoutError`, which is retried
`fn` is not a function (e.g. a promise)	Rejects with `TypeError`
`retries` is negative / not an integer	Rejects with `RangeError`

This table is the implementation checklist and the test file.

---

3. Design the architecture

This is a utility, so "architecture" means small pure functions with one job each:

Unit	Responsibility
`retry(fn, options)`	The loop: call, catch, decide, wait, repeat
`getDelay(attempt, config)`	Pure math: backoff + cap + jitter
`sleep(ms, signal)`	Promise-based wait that can be cancelled

Unit	Responsibility
`withTimeout(promise, ms)`	Race a promise against a timer
`TimeoutError`	Distinct error type so callers can `instanceof` it

```
retry(fn, options)
 ├── fn(attempt, signal)            ← user code
 ├── withTimeout(promise, ms)       ← only if options.timeout
 ├── shouldRetry(err, attempt)      ← user policy
 ├── getDelay(attempt, config)      ← pure, easy to unit test
 ├── onRetry(err, attempt, wait)    ← user hook
 └── sleep(wait, signal)            ← cancellable wait
```

Start with only `retry` + a fixed `setTimeout`. Pull `getDelay`, `sleep`, `withTimeout` out only when each gets its own logic.

---

4. Design the state / data model

There is no UI state — the "state" is what the loop must remember between attempts.

State or field	Purpose
`attempt`	1-based number of the current call; drives backoff and the stop condition
`err` (last error)	What we reject with when we give up
`wait`	Delay before the next attempt, computed per attempt
timer id (inside `sleep`)	So abort can `clearTimeout` it
`options` (read-only)	`retries, delay, factor, maxDelay, jitter, timeout, shouldRetry, onRetry, signal`

```
const {
  retries = 3,
  delay = 1000,
  factor = 2,
  maxDelay = 30_000,
  jitter = false,
  timeout,
  shouldRetry = () => true,
  onRetry,
  signal,
} = options;
```

```
for (let attempt = 1; ; attempt++) { /* ... */ }
```

Senior note

- **Derived, not stored:** the wait is computed from `attempt` each time — no `currentDelay` variable that could drift. "Attempts left" is `retries + 1 - attempt`; don't keep a second counter.
- **We don't store an `errors[]` array** unless the caller asks for all errors — the last error is enough and avoids memory growth.
- **Ownership:** `retry` owns `attempt` and the timer. The **caller** owns the `AbortController` — `retry` only listens to the signal, it never aborts it.

---

5. Design the data structures and contracts

Function contract

```
retry<T>(
  fn: (attempt: number, signal?: AbortSignal) => Promise<T> | T,
  options?: {
    retries?: number;      // extra attempts, default 3  → 4 calls max
    delay?: number;        // ms before 1st retry, default 1000
    factor?: number;       // backoff multiplier, default 2 (1 = fixed delay)
    maxDelay?: number;     // cap, default 30000
    jitter?: boolean;      // random wait in [0, delay], default false
    timeout?: number;      // ms per attempt, default none
    shouldRetry?: (err, attempt) => boolean | Promise<boolean>;
    onRetry?: (err, attempt, wait) => void;
    signal?: AbortSignal;
  }
): Promise<T>
```

- **Resolves** with the first successful value.
- **Rejects** with: the last error · `signal.reason` if aborted · `TypeError` / `RangeError` on bad input.

Example data (a mock flaky API)

```
// Fails `failures` times, then succeeds — deterministic, so tests are repeatable.
function flaky(failures) {
  const f = async () => {
    f.calls++;
    if (f.calls <= failures) throw new Error(`fail ${f.calls}`);
    return 'ok';
  };
```

```
  f.calls = 0;
  return f;
}
```

Why deterministic instead of `Math.random() < 0.5`? Tests must give the same result every run. Random failure is fine for a demo, never for a test.

Real usage

```
const orders = await retry(
  async (attempt, signal) => {
    const res = await fetch('/api/orders', { signal });
    if (!res.ok) {
      const err = new Error(`HTTP ${res.status}`);
      err.status = res.status;
      throw err;
    }
    return res.json();
  },
  {
    retries: 4,
    delay: 500,
    jitter: true,
    timeout: 5000,
    // network error, 5xx, 429 → retry; 400/401/404 → give up now
    shouldRetry: (err) => !err.status || err.status >= 500 || err.status === 429,
    onRetry: (err, attempt, wait) => console.warn(`attempt ${attempt} failed: ${err.message},
retrying in ${wait}ms`),
  }
);
```

---

6. Trace the data flow

Scenario: `fn` fails once with a 503, then succeeds. `retries: 3, delay: 500`.

```
retry(fn, opts)
   │
   ▼
validate input (fn is a function, retries ≥ 0)
   │
   ▼
attempt = 1 → signal aborted? no
```

```
   |
   ▼
fn(1, signal) → Promise rejects with Error("HTTP 503")
   |
   ▼
catch: signal aborted? no · attempt(1) > retries(3)? no · shouldRetry(err)? yes
   |
   ▼
wait = getDelay(1) = 500 * 2^0 = 500 ms
   |
   ▼
onRetry(err, 1, 500)  → log "attempt 1 failed, retrying in 500ms"
   |
   ▼
await sleep(500, signal)        (timer registered, abort listener registered)
   |
   ▼
attempt = 2 → fn(2, signal) → resolves { orders: [...] }
   |
   ▼
return value → retry's promise resolves with { orders: [...] }
```

---

7. Important technical challenges

Challenge 1 — A promise can't be retried; a function can

Problem. A candidate writes `retry(fetch('/api'), 3)`. `fetch` has **already run** when `retry` receives the promise. Awaiting the same rejected promise three times returns the same rejection three times — no new request is made.

Solution. Accept a **factory** — a function that creates a fresh promise on every call. Validate it, so the mistake fails loudly.

```
if (typeof fn !== 'function') {
  throw new TypeError('retry expects a function that returns a promise');
}
// usage: retry(() => fetch('/api'))   ✅     retry(fetch('/api'))   ❌
```

Challenge 2 — Sequential attempts without recursion bugs or stack growth

Problem. The classic recursive version (`.catch(() => retry(fn, n - 1))`) works, but it is easy to forget a `return` (the outer promise never settles), to leak an unhandled rejection, or to mis-count attempts. Synchronous throws from `fn` also escape a `.then` chain that starts with `fn()`.

Solution. An `async` loop with `try/catch`. Calling `fn` **inside** `try` catches sync throws too. `Promise.resolve(...)` normalises plain values. Each `await` frees the stack, so 1,000 retries do not grow it.

```
for (let attempt = 1; ; attempt++) {
  try {
    return await Promise.resolve(fn(attempt, signal));
  } catch (err) {
    if (attempt > retries || !(await shouldRetry(err, attempt))) throw err;
    await sleep(getDelay(attempt, config), signal);
  }
}
```

The recursive version, for comparison (expect to be asked for it):

```
function retryBasic(fn, retries = 3, delay = 1000) {
  return new Promise((resolve, reject) => {
    const attempt = (left) => {
      Promise.resolve()
        .then(fn)                    // .then(fn) also catches sync throws
        .then(resolve)
        .catch((err) => {
          if (left === 0) return reject(err);
          setTimeout(() => attempt(left - 1), delay);
        });
    };
    attempt(retries);
  });
}
```

Challenge 3 — Backoff, cap and jitter (don't DDoS your own server)

Problem. Server goes down for 2 s. 10,000 clients retry every 1,000 ms on a fixed delay → every second the recovering server gets 10,000 requests in the same instant and falls over again (**thundering herd**). Pure exponential backoff without a cap waits 2^20 seconds on attempt 20.

Solution. `delay * factor^(attempt-1)`, clamp with `maxDelay`, and optionally pick a random value in `[0, base]` ("full jitter") so clients spread out.

```
export function getDelay(attempt, { delay, factor, maxDelay, jitter }) {
  const base = Math.min(delay * factor ** (attempt - 1), maxDelay);
  return jitter ? Math.round(Math.random() * base) : base;
}
// delay 500, factor 2 → 500, 1000, 2000, 4000 ... capped at maxDelay
```

Keeping it a **pure function** means it can be tested without timers.

Challenge 4 — Cancellation and per-attempt timeout

Problem. User leaves the page while we are in a 16-second backoff wait. Without cancellation the timer keeps running, the next attempt fires, and the result is written into an unmounted component. Separately, one attempt can hang forever (no response, no error) and the retry never happens.

Solution. (a) A `sleep` that listens to an `AbortSignal`, clears its timer and rejects immediately. Check `signal.aborted` before each attempt and after each failure. Pass the signal into `fn` so `fetch` can abort too. (b) `Promise.race` the attempt against a timer that rejects with `TimeoutError`, and clear the timer in `finally`.

```
export function sleep(ms, signal) {
  return new Promise((resolve, reject) => {
    if (signal?.aborted) return reject(signal.reason);
    const onAbort = () => { clearTimeout(id); reject(signal.reason); };
    const id = setTimeout(() => {
      signal?.removeEventListener('abort', onAbort);   // no listener leak
      resolve();
    }, ms);
    signal?.addEventListener('abort', onAbort, { once: true });
  });
}

export function withTimeout(promise, ms) {
  let id;
  const timer = new Promise((_, reject) => {
    id = setTimeout(() => reject(new TimeoutError(ms)), ms);
  });
  return Promise.race([promise, timer]).finally(() => clearTimeout(id));
}
```

Note: `Promise.race` stops **waiting** for a slow attempt — it does not stop the work. To really stop it, `fn` must honour the `signal` it receives.

---

8. Implement in the correct order

#	Step	Working result after this step
1	`retry(fn, retries)` with an `async` loop, no delay	Retries N times, rejects with last error
2	Write `flaky(n)` mock + first 4 asserts	Tests for success / fail-then-succeed / always-fail / `retries: 0`

#	Step	Working result after this step
3	Fixed `delay` with `await new Promise(r => setTimeout(r, ms))`	Waits between attempts
4	Input validation (`TypeError` , `RangeError`)	Wrong usage fails loudly
5	Extract `getDelay` : `factor` + `maxDelay`	Exponential backoff, capped
6	`jitter` option in `getDelay`	Spread-out retries
7	`shouldRetry(err, attempt)` (sync or async)	Skip retry on 4xx
8	`onRetry(err, attempt, wait)` hook	Logging / metrics, and tests can read the waits
9	Cancellable `sleep(ms, signal)` + `signal.aborted` checks	Abort stops retries at once
10	`withTimeout` + `TimeoutError`	Hanging attempt becomes a retryable failure
11	Pass `(attempt, signal)` into `fn`	`fetch` can be aborted too
12	Run the full test table out loud	Every row of section 2 verified

---

9. Cover all states and edge cases

State / case	How it is handled
Initial — first call	`attempt = 1` , no delay before the first call
Processing — attempt in flight	`await` on `fn` 's promise (optionally raced with timeout)
Waiting — between attempts	`sleep(wait, signal)` — one timer, cancellable
Success	Return the value — resolves immediately, remaining attempts skipped
Error — retries exhausted	Rethrow the last error unchanged (stack and `status` kept)
Error — not retryable	`shouldRetry` returns false → rethrow immediately
Cancelled	Reject with `signal.reason` (an `AbortError` `DOMException` by default)
Timed out attempt	`TimeoutError` , goes through `shouldRetry` like any other error
`retries: 0`	Behaves like a plain call with error passthrough
`fn` returns a non-promise	`Promise.resolve` wraps it
`fn` throws synchronously	Caught by `try` — same as rejection
`fn` rejects with `undefined` / a string	Rethrown as-is — we never read `err.message` in the core
`shouldRetry` itself throws	Its error propagates — a bug in policy should be visible, not retried
`onRetry` throws	Propagates and stops retrying (documented; could be wrapped in try/catch)

State / case	How it is handled
`delay: 0`	Retries on the next macrotask — no sync busy loop
Abort while `fn` is running	Next check after the failure rejects with the reason; `fn` gets the signal to abort itself
Very large `retries`	Constant stack (loop + `await`), `maxDelay` keeps waits bounded

---

10. Final structure

```
Implement Auto-retry promises/
├── retry.js          # retry, getDelay, sleep, withTimeout, TimeoutError
├── retry.test.js     # node:assert tests — one per row of the use-case table
└── index.js          # demo: retry a flaky fetch-like function and log attempts
```

---

11. Full working code

retry.js

```
// retry.js — auto-retry any promise-returning function

export class TimeoutError extends Error {
  constructor(ms) {
    super(`Attempt timed out after ${ms}ms`);
    this.name = 'TimeoutError';
  }
}

// Wait `ms`, but reject immediately (and clear the timer) if the signal aborts.
export function sleep(ms, signal) {
  return new Promise((resolve, reject) => {
    if (signal?.aborted) return reject(signal.reason);
    const onAbort = () => {
      clearTimeout(id);
      reject(signal.reason);
    };
    const id = setTimeout(() => {
      signal?.removeEventListener('abort', onAbort);
      resolve();
    }, ms);
    signal?.addEventListener('abort', onAbort, { once: true });
```

```
  });
}

// Reject with TimeoutError if `promise` takes longer than `ms`.
export function withTimeout(promise, ms) {
  let id;
  const timer = new Promise((_, reject) => {
    id = setTimeout(() => reject(new TimeoutError(ms)), ms);
  });
  return Promise.race([promise, timer]).finally(() => clearTimeout(id));
}

// Delay before retry number `attempt` (1-based): delay * factor^(attempt-1), capped.
export function getDelay(attempt, { delay, factor, maxDelay, jitter }) {
  const base = Math.min(delay * factor ** (attempt - 1), maxDelay);
  return jitter ? Math.round(Math.random() * base) : base; // "full jitter"
}

export async function retry(fn, options = {}) {
  const {
    retries = 3,
    delay = 1000,
    factor = 2,
    maxDelay = 30_000,
    jitter = false,
    timeout,
    shouldRetry = () => true,
    onRetry,
    signal,
  } = options;

  if (typeof fn !== 'function') {
    throw new TypeError('retry expects a function that returns a promise');
  }
  if (!Number.isInteger(retries) || retries < 0) {
    throw new RangeError('retries must be a non-negative integer');
  }

  for (let attempt = 1; ; attempt++) {
    if (signal?.aborted) throw signal.reason;
    try {
      // fn() inside try: a synchronous throw is caught like a rejection
      const result = Promise.resolve(fn(attempt, signal));
      return await (timeout ? withTimeout(result, timeout) : result);
    } catch (err) {
```

```
      if (signal?.aborted) throw signal.reason;
      if (attempt > retries || !(await shouldRetry(err, attempt))) throw err;
      const wait = getDelay(attempt, { delay, factor, maxDelay, jitter });
      onRetry?.(err, attempt, wait);
      await sleep(wait, signal);
    }
  }
}
```

retry.test.js

```
// retry.test.js — run with: node retry.test.js
import assert from 'node:assert/strict';
import { retry, getDelay, TimeoutError } from './retry.js';

// Fails `failures` times, then resolves with "ok". Counts calls.
function flaky(failures) {
  const f = async () => {
    f.calls++;
    if (f.calls <= failures) throw new Error(`fail ${f.calls}`);
    return 'ok';
  };
  f.calls = 0;
  return f;
}

const fast = { delay: 5 };

// 1. succeeds first time -> one call
let fn = flaky(0);
assert.equal(await retry(fn, fast), 'ok');
assert.equal(fn.calls, 1);

// 2. fails twice, then succeeds
fn = flaky(2);
assert.equal(await retry(fn, { ...fast, retries: 3 }), 'ok');
assert.equal(fn.calls, 3);

// 3. always fails -> 1 + retries calls, rejects with the LAST error
fn = flaky(Infinity);
await assert.rejects(retry(fn, { ...fast, retries: 2 }), { message: 'fail 3' });
assert.equal(fn.calls, 3);

// 4. retries: 0 -> exactly one attempt
```

```
fn = flaky(Infinity);
await assert.rejects(retry(fn, { retries: 0 }), { message: 'fail 1' });
assert.equal(fn.calls, 1);

// 5. shouldRetry=false stops immediately
fn = flaky(Infinity);
await assert.rejects(retry(fn, { ...fast, shouldRetry: () => false }));
assert.equal(fn.calls, 1);

// 6. synchronous throw is retried too
let syncCalls = 0;
const syncThrow = () => {
  if (++syncCalls < 2) throw new Error('sync');
  return 'done';
};
assert.equal(await retry(syncThrow, fast), 'done');

// 7. backoff delays reported to onRetry: 10, 20, 40 (capped at maxDelay)
const waits = [];
await assert.rejects(
  retry(flaky(Infinity), { retries: 4, delay: 10, maxDelay: 30, onRetry: (_e, _a, w) =>
waits.push(w) })
);
assert.deepEqual(waits, [10, 20, 30, 30]);

// 8. jitter stays within [0, base]
for (let i = 0; i < 100; i++) {
  const d = getDelay(3, { delay: 100, factor: 2, maxDelay: 1e9, jitter: true });
  assert.ok(d >= 0 && d <= 400);
}

// 9. abort during the wait -> rejects quickly, no more attempts
fn = flaky(Infinity);
const ac = new AbortController();
const start = Date.now();
setTimeout(() => ac.abort(new Error('cancelled')), 20);
await assert.rejects(retry(fn, { delay: 1000, signal: ac.signal }), { message: 'cancelled' });
assert.equal(fn.calls, 1);
assert.ok(Date.now() - start < 500);

// 10. already-aborted signal -> fn never called
fn = flaky(0);
await assert.rejects(retry(fn, { signal: AbortSignal.abort() }));
assert.equal(fn.calls, 0);
```

```
// 11. per-attempt timeout -> TimeoutError, and it is retried
let slowCalls = 0;
const slowThenFast = () => (++slowCalls === 1 ? new Promise((r) => setTimeout(() => r('late'),
200)) : 'fast');
assert.equal(await retry(slowThenFast, { ...fast, timeout: 20 }), 'fast');
await assert.rejects(
  retry(() => new Promise(() => {}), { ...fast, retries: 1, timeout: 10 }),
  TimeoutError
);

// 12. bad input
await assert.rejects(retry(Promise.resolve(1)), TypeError);
await assert.rejects(retry(fn, { retries: -1 }), RangeError);

console.log('all retry tests passed');
```

index.js (demo)

```
import { retry } from './retry.js';

// Fake API: fails twice with 503, then succeeds.
let calls = 0;
const fakeApi = async () => {
  calls++;
  if (calls < 3) {
    const err = new Error('Service Unavailable');
    err.status = 503;
    throw err;
  }
  return { orders: [{ id: 'o-1', total: 499 }] };
};

const data = await retry(fakeApi, {
  retries: 4,
  delay: 300,
  shouldRetry: (err) => !err.status || err.status >= 500 || err.status === 429,
  onRetry: (err, attempt, wait) =>
    console.log(`attempt ${attempt} failed (${err.message}) → retrying in ${wait}ms`),
});

console.log('result:', data);
// attempt 1 failed (Service Unavailable) → retrying in 300ms
// attempt 2 failed (Service Unavailable) → retrying in 600ms
// result: { orders: [ { id: 'o-1', total: 499 } ] }
```

Run with `node retry.test.js` (add `"type": "module"` in `package.json`, or rename to `.mjs`).

---

12. Wrap-up

Edge cases handled

Sync throws, non-promise returns, `retries: 0`, non-retryable errors, exponential backoff with cap, jitter, cancellation before / during waits, per-attempt timeout, bad input, no unhandled rejections, no leaked timers or abort listeners.

Trade-offs to mention

- **Last error vs all errors** — last error is simple and keeps the original type; an `AggregateError` gives full history but changes what callers catch.
- `Promise.race` **timeout** — guarantees `retry` moves on, but cannot stop the underlying work unless `fn` uses the signal.
- **Retry everything by default** — friendly for a utility, but dangerous for non-idempotent calls (`POST /payments`). In production default `shouldRetry` to network/5xx only.
- **Full jitter** — best for spreading load, but makes individual waits unpredictable (and tests need `jitter: false`).
- `async` **loop vs recursion** — loop is easier to read, count and debug; recursion is what many interviewers expect first, so know both.

With more time

- Honour the `Retry-After` header for 429 / 503 (`shouldRetry` could return a delay).
- A `withRetry(fn, opts)` wrapper that returns a retrying version of any function.
- Overall deadline (`maxTime`) in addition to per-attempt timeout.
- Fake timers (`node:test` mock timers / Jest) so tests run in 0 ms.
- Circuit breaker on top: stop calling a service that keeps failing.
- TypeScript types and a retry budget shared across calls (limit retry traffic to e.g. 10% of requests).

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why do you take a function and not a promise?

A promise represents work that has **already started**. Re-awaiting a rejected promise just gives the same rejection again. A function lets us start a **new** attempt every time: `retry(() => fetch(url))`.

Q2. Does `retries: 3` mean 3 or 4 calls?

In this design, 3 **retries** = 4 calls (1 try + 3 retries). That's the convention in p-retry and most HTTP clients. I confirmed it with the clarifying question and the test `retries: 2 → 3 calls`.

Q3. Which error do you reject with, and why?

The last one — it reflects the current state of the service. If the interviewer wants all of them, collect into an array and `throw new AggregateError(errors, 'All attempts failed')`.

Q4. Should every error be retried?

No. Retry transient errors: network failures, timeouts, 5xx, 429. Don't retry 400 / 401 / 403 / 404 — they will fail the same way again. That's why `shouldRetry` exists.

State design

Q5. Why don't you store `currentDelay` and multiply it each time?

Because it can be **derived** from `attempt`. One source of truth means no bug where the delay and the counter disagree, and `getDelay` stays a pure function that is trivial to test.

Q6. Who owns the AbortController?

The caller. `retry` only reads the signal. If `retry` created its own controller, the caller would have no way to cancel; if `retry` aborted the caller's controller, it could cancel unrelated work sharing that signal.

Hard parts

Q7. What happens if `fn` throws synchronously?

`fn` is called inside the `try` block, so the throw is caught exactly like a rejection and retried. In the recursive version, `Promise.resolve().then(fn)` gives the same guarantee.

Q8. What is jitter and why do you need it?

Randomness added to the wait. Without it, all clients that failed at the same moment retry at the same moment, creating spikes that keep the server down (thundering herd). Full jitter: `Math.random() * base`.

Q9. Why cap the delay with `maxDelay`?

Exponential growth gets absurd fast: `1000 * 2^15` ≈ 9 hours. A cap keeps the worst-case wait reasonable and predictable.

Q10. Does your timeout cancel the slow request?

No — `Promise.race` only stops **waiting**. The request continues in the background. To really cancel, `fn` must use the signal; a better version gives each attempt its own signal: `AbortSignal.any([signal, AbortSignal.timeout(ms)])`.

Q11. Can the losing promise in `Promise.race` cause an unhandled rejection?

No. `Promise.race` attaches handlers to every input promise, so a later rejection of the slow attempt is handled (and ignored). The timer promise is also cleared in `finally`.

Q12. How do you avoid leaking timers and listeners?

`sleep` removes the abort listener when the timer fires, and clears the timer when abort fires. `withTimeout` clears its timer in `finally`. `{ once: true }` on the listener is a second safety net.

Implementation steps

Q13. Write it recursively without async/await.

```
const retryP = (fn, retries, delay) =>
  Promise.resolve().then(fn).catch((err) =>
    retries === 0
      ? Promise.reject(err)
      : new Promise((r) => setTimeout(r, delay)).then(() => retryP(fn, retries - 1, delay))
  );
```

Q14. Can the recursive version blow the stack with 10,000 retries?

No. Each recursive call happens inside a `.then` / `setTimeout` callback, i.e. a fresh call stack. It does create a chain of promises, which costs some memory, while the loop version keeps memory constant.

Q15. How would you test this without waiting real seconds?

Use tiny delays (5–20 ms) like the test file, or fake timers (`jest.useFakeTimers()` / `mock.timers` in `node:test`) and advance time manually. Keep `getDelay` pure and test it directly with no timers at all.

Performance and "what if"

Q16. What if this runs on 1 million clients and the server goes down?

Retries multiply load exactly when the server is weakest. Use exponential backoff + full jitter + `maxDelay`, respect `Retry-After`, limit retries (2–3), and add a circuit breaker so clients stop calling for a while after repeated failures.

Q17. What if the API is slow but not failing?

Without a timeout, one attempt can hang forever and we never retry. Add a per-attempt `timeout` and an overall deadline. Choose the timeout from real latency data (e.g. p99 + margin), not a guess.

Q18. What if the operation is not idempotent — e.g. "charge card"?

A retry after a timeout may charge twice, because the first request might have succeeded. Either don't retry, or send an **idempotency key** (same key on every attempt) so the server deduplicates.

Q19. What if the user navigates away in a React app?

Create an `AbortController` in `useEffect`, pass `signal` to `retry`, and call `controller.abort()` in the cleanup function. `retry` rejects immediately and no state update happens on an unmounted component.

Trade-offs / "why not X?"

Q20. Why not `setInterval` for the retries?

The interval fires on a fixed clock, regardless of whether the previous attempt finished — attempts can overlap. We need the next attempt to start only **after** the previous one fails and the wait completes. `setTimeout` per wait gives that.

Q21. Why not run all attempts in parallel with `Promise.any`?

That sends N requests at once — the opposite of backing off — and multiplies server load. `Promise.any` is for **hedging** (race several replicas), a different pattern from retrying.

Q22. Why an options object instead of positional arguments?

`retry(fn, 3, 1000, 2, 30000, true)` is unreadable and every new feature breaks the signature. Named options are self-documenting, have defaults, and new options are backward compatible.